

Game Recommender — Quick Documentation

1. Project Goal

A MyAnimeList-style game tracking/review site with personalized recommendations.

Users can:

- Track games they've played / want to play
- Rate and review games
- Maintain a personal library
- Receive personalized game recommendations
- Potentially link to Steam/new releases/sales later

The recommendation system starts as **content-based filtering** and can later be expanded into collaborative/hybrid filtering.

2. Game Object

Each game is stored as an object.

```
{
  id: 1,
  name: "Cyberpunk 2077",
  tags: ["Action", "RPG", "Open World"],
  platforms: ["PC"]
}
```

Example game list:

```
const games = [
  {
    id: 1,
    name: "Deadlock",
    tags: ["Action", "Shooter", "Multiplayer", "MOBA"],
    platforms: ["PC"]
  },
  {
    id: 2,
    name: "Cyberpunk 2077",
```

```

    tags: ["Action", "RPG", "Open World"],
    platforms: ["PC"]
  },

  {
    id: 3,
    name: "Baldur's Gate 3",
    tags: ["RPG", "Fantasy", "Turn-Based", "Story"],
    platforms: ["PC", "PS5", "Xbox"]
  },

  {
    id: 4,
    name: "Elden Ring",
    tags: ["RPG", "Action", "Open World", "Fantasy"],
    platforms: ["PC", "PS5", "Xbox"]
  }
];

```

3. User Library Object

The library should reference games using their ID instead of duplicating the entire game object.

```

{
  gameId: 2,
  status: "completed",
  rating: 9,
  review: "Great open-world RPG."
}

```

Possible statuses:

```

"wantToPlay"
"playing"
"completed"
"dropped"
"onHold"

```

Example:

```
const testLibrary = [  
  {  
    gameId: 1,  
    status: "completed",  
    rating: 7  
  },  
  
  {  
    gameId: 2,  
    status: "completed",  
    rating: 9  
  }  
];
```

This means the user has already interacted with Deadlock and Cyberpunk 2077.

4. User Preferences

Preferences represent how much the user appears to like/dislike each tag.

```
const userPreferences = {  
  RPG: 10,  
  Action: 5,  
  Fantasy: 8,  
  "Open World": 7,  
  Shooter: -2  
};
```

Positive number = user likes the tag.

Negative number = user dislikes the tag.

Higher number = stronger preference.

5. Generating Preferences From Ratings

Instead of manually creating preferences, the system can eventually learn them from the user's ratings.

For a 1–10 rating:

```
Rating 10 → +5
Rating 9  → +4
Rating 8  → +3
Rating 7  → +2
Rating 6  → +1
Rating 5  → 0
Rating 4  → -1
Rating 3  → -2
Rating 2  → -3
Rating 1  → -4
```

Basic formula:

```
preferenceValue = rating - 5;
```

Every tag belonging to the rated game receives that contribution.

Example:

```
Cyberpunk 2077 = 9/10
```

```
Tags:
Action
RPG
Open World
```

Adds:

```
Action      +4
RPG          +4
Open World   +4
```

If the user then rates another RPG highly, the RPG preference increases again.

6. Recommendation Score

Each candidate game receives a score based on its tags.

Example preferences:

```
const userPreferences = {  
  RPG: 10,  
  Fantasy: 8,  
  "Open World": 6,  
  Action: 3,  
  Shooter: -2  
};
```

Game:

```
Elden Ring  
Tags:  
RPG  
Action  
Open World  
Fantasy
```

Score:

RPG	+10
Action	+3
Open World	+6
Fantasy	+8

Total	27

So:

```
Elden Ring → 27
```

A higher score means the game matches the user's preferences more strongly.

7. Recommendation Pipeline

The current algorithm works approximately like this:

```
User ratings  
  ↓  
Game tags  
  ↓
```

```
Build user preferences
  ↓
Score every game
  ↓
Remove games already in library
  ↓
Sort by score
  ↓
Return highest-scoring games
```

Example:

Game	Score

Elden Ring	27
Baldur's Gate 3	26
Deadlock	1
Some Sports Game	-5

If Elden Ring is already in the user's library, remove it:

1. Baldur's Gate 3 – 26
2. Deadlock – 1
3. Some Sports Game – -5

8. Recommendation Result Object

The recommender should keep the game and its score together.

```
{
  game: game,
  score: 26
}
```

Example:

```
[
  {
    game: {
      id: 3,
```

```

        name: "Baldur's Gate 3",
        tags: ["RPG", "Fantasy", "Turn-Based", "Story"],
        platforms: ["PC", "PS5", "Xbox"]
    },
    score: 26
},

{
  game: {
    id: 4,
    name: "Elden Ring",
    tags: ["RPG", "Action", "Open World", "Fantasy"],
    platforms: ["PC", "PS5", "Xbox"]
  },
  score: 25
}
]

```

The frontend can then display:

Recommended for You

Baldur's Gate 3

Match Score: 26

Elden Ring

Match Score: 25

9. Normalization — Later

Raw preference values can become very large as users rate more games.

Example:

User A:

RPG = 10

User B:

RPG = 147

This doesn't necessarily mean User B likes RPGs 14.7× more.

Eventually we should **normalize** preferences/scores so they remain comparable.

This is intentionally a later step. First get the basic recommendation algorithm working.

10. Future: Collaborative Filtering

Content-based filtering asks:

"What games match THIS user's preferences?"

Collaborative filtering asks:

"What games do people with similar tastes like?"

Example:

```
User A:  
Cyberpunk → 9  
Elden Ring → 10  
BG3 → 9
```

```
User B:  
Cyberpunk → 10  
Elden Ring → 9  
BG3 → 10
```

Because their ratings are similar, User A and User B are considered similar users.

If User B likes a game User A hasn't played:

```
User B → Hades → 9  
User A → Hades → ?
```

Hades becomes a recommendation for User A.

This can eventually be combined with the content-based system.

11. Future: Hybrid Recommendations

Eventually:

```
Content-Based Score
  +
Collaborative Score
  ↓
Hybrid Score
  ↓
Recommendation
```

For example:

```
60% Content-Based
40% Collaborative
```

```
finalScore =
  contentScore * 0.6 +
  collaborativeScore * 0.4;
```

This should come after the basic content-based system is solid.

12. Suggested MERN Data Models

Game

```
{
  _id: ObjectId,
  name: String,
  tags: [String],
  platforms: [String],
  releaseDate: Date,
  description: String
}
```

User

```
{
  _id: ObjectId,
  username: String,
  email: String
}
```

Library Entry

```
{
  userId: ObjectId,
  gameId: ObjectId,
  status: String,
  rating: Number,
  review: String
}
```

Recommendation

Recommendations don't necessarily need to be permanently stored.

They can be calculated when requested:

```
GET /api/recommendations/:userId
```

Response:

```
[
  {
    gameId: 4,
    score: 27
  },
  {
    gameId: 3,
    score: 24
  }
]
```

The backend can then retrieve the game information and send it to React.

13. Division of Work

Recommendation/Algorithm Side

Responsible for:

```
Preference generation
↓
```

```
Game scoring
  ↓
Library filtering
  ↓
Ranking
  ↓
Future normalization
  ↓
Future collaborative filtering
```

MERN Side

Responsible for:

```
React UI
  ↓
Express API
  ↓
MongoDB
```

Including:

- User accounts
- Game pages
- Reviews
- Library management
- Game search
- API routes
- Database models
- Frontend recommendation display

The two sides communicate through API endpoints.

Current Milestone

The first version only needs:

```
Game data
+
User preferences
+
User library
```



Content-based recommendations

Do **not** worry about machine learning, collaborative filtering, Steam integration, or normalization yet.

Get the simple recommender working first, then build complexity on top of it.